

SoloMuse architecture recommendations for a three-layer controllable music generation pipeline

Executive summary

The strongest *scientifically defensible* direction for SoloMuse in 2026 is to treat the Renderer as **conditional generative modeling in neural-codec token space**, and to redesign the other two layers to produce (and consume) conditioning signals that are *predictive, calibrated, and hard for the Renderer to trivially ignore*. This recommendation is anchored in the dominant open research line for high-fidelity music/audio generation: **neural audio codecs (SoundStream/EnCodec/DAC-style) + sequence models (Transformer families; sometimes hierarchical) + optional diffusion decoding/post-filtering.** ¹

Best overall integrated stack (quality first): - **Renderer representation:** switch `renderer_target.npy` from continuous WaveChunkCodec to **codec tokens** (recommended default: **EnCodec-family at 24 kHz** for direct compatibility with your data; optional experiment: **DAC** if you can afford resampling + higher token rate). Token LMs dominate open SOTA pipelines such as MusicGen (single-stage Transformer LM over EnCodec tokens) and MusicLM/AudioLM (hierarchical semantic→acoustic token generation). ² - **Renderer model: hierarchical codec-token generation** patterned after AudioLM/MusicLM: generate *coarse* acoustic codebooks first, then *fine* codebooks, decoding via the codec decoder; optionally add a **diffusion decoder** (e.g., Multi-Band Diffusion) as a high-fidelity “renderer head” if artifacts remain. ³ - **Situation encoder:** use a music-specialized pretrained representation model (top pick: **MERT**) and learn a small projection to your `[32]` situation vector, optionally with *task-driven* fine-tuning to maximize downstream utility for Intent/Renderer. ⁴ - **Intent planner:** replace “simple regression head” thinking with a **cross-attention planning model**: 60 query steps (10 Hz) cross-attend into higher-rate audio latents/tokens; inject Situation via FiLM/adapters; train with MSE/Huber plus temporal regularizers, and consider a probabilistic head or diffusion/flow for multimodality once the baseline is stable. (The “local control” framing is strongly supported by temporally controlled music work like JASCO, even though its target domain is text-to-music.) ⁵

Practical fallback integrated stack (minimize risk / keep current artifacts longer): - Keep your current `intent_targets.npy` and `renderer_target.npy`, but replace Conv1D render regression with a **U-Net-style 1D/2D hybrid** (time × feature) and/or **diffusion in the WaveChunkCodec space** (fast to prototype, better multimodal fit than pure MSE regression); keep Situation/Intent upgrades as above. This mirrors the broader diffusion trend in audio generation and “diffusion post-filter” approaches used to reduce codec artifacts. ⁶

The rest of this report explains *why* each architecture family fits (or does not fit) each layer, then provides ranked candidates and a coherent end-to-end training/evaluation plan.

Architecture landscape by layer

SoloMuse's three layers map cleanly onto three different problem classes: **global representation learning (Situation)**, **temporally aligned low-rate regression/planning (Intent)**, and **high-fidelity conditional generation (Renderer)**. The “best” architecture family shifts accordingly.

Transformers and hierarchical Transformers

Where they shine (audio/music evidence): Transformer LMs over discrete audio tokens are the backbone of modern open, high-quality music generation systems. MusicGen demonstrates a single-stage Transformer LM over multiple codebook streams of EnCodec tokens and highlights that *codebook interleaving patterns* materially affect quality/compute trade-offs. ⁷ MusicLM and AudioLM push this further with **hierarchical token modeling** (semantic→acoustic, plus coarse→fine acoustic), explicitly to improve long-term structure and fidelity. ⁸

Fit to SoloMuse layers: - **Situation:** Transformers are excellent *encoders* when pooled (CLS/attention pooling) to a single embedding, especially in BERT-style masked objectives on audio features; MERT is an example of a music-specialized masked-model approach, using teacher signals to stabilize and improve music-relevant representation learning. ⁹ - **Intent:** A Transformer encoder-decoder or Perceiver-style cross-attention is a near-perfect fit because your output is a 60-step sequence that must align to an input audio segment that may be represented at higher frame rate (e.g., 50 Hz codec frames). Cross-attention cleanly handles rate mismatch and controllability. - **Renderer:** Transformer decoders (autoregressive) remain the most validated open approach for codec-token music generation (MusicGen) and for hierarchical multi-stage generation (MusicLM/AudioLM). ¹⁰

Typical representations: codec tokens at tens of Hz with multiple residual codebooks (e.g., MusicGen's EnCodec configuration at 50 Hz with 4 quantizers of size 2048; MusicLM's SoundStream setup at 50 Hz with 12×1024 acoustic quantizers). ¹¹

Stability/data-efficiency/latency: - Training is generally stable *once tokenization is stable*, but depends on the codec quality; tokenizers themselves can be unstable (codebook collapse, adversarial loss balancing). ¹² - AR inference cost scales with token count; interleaving patterns can reduce steps at some quality cost and are explicitly studied in MusicGen. ¹³ - Controllability is excellent via cross-attention, FiLM/adapters, or conditional dropout (for guidance-like inference). ¹⁴

Conformers and hybrid conv-attention encoders

Conformer was proposed to combine convolution (local patterns) with self-attention (global dependencies) for audio sequences and is widely viewed as a strong general-purpose audio encoder backbone. ¹⁵

Fit to layers: - **Situation:** strong as an encoder to produce a pooled embedding, especially if you want robust local rhythmic/timbral cues and global structure. - **Intent:** strong when you want stable training and locality bias across time while still modeling global phrasing; less necessary at only 60 steps unless your conditioning stream is high-rate and noisy. - **Renderer:** useful mainly as an *encoder* for conditioning (backing/context audio), not as the core generative model, where codec-token LMs have stronger supporting evidence in music generation.

State-space sequence models (S4, Mamba family)

Structured state-space models like S4 target long sequences efficiently, and Mamba proposes a selective state-space architecture with linear-time scaling and claims strong results across modalities including audio and fast inference throughput. ¹⁶

Fit to layers: - **Situation:** not compelling; sequence length is short (6 s) and pooling is easy. - **Intent:** possible but unnecessary at 60 steps. - **Renderer:** relevant as a *trade-off* backbone for long codec-token sequences if AR Transformer inference becomes a bottleneck. Evidence for *music generation quality parity* with Transformers is still weaker in open literature than for Transformers; treat as “promising but higher uncertainty.” ¹⁷

Diffusion and flow-matching models

Diffusion models are well-validated for high-fidelity waveform generation (e.g., DiffWave, WaveGrad) and for latent audio generation frameworks (e.g., AudioLDM2). ¹⁸ In the codec context, Multi-Band Diffusion explicitly targets decoding from discrete tokens to higher-fidelity audio and reports perceptual improvements at equal bitrate. ¹⁹ Stable Audio Open is an example of a Transformer-based diffusion model operating in an autoencoder latent space (DiT-style) for text-to-audio. ²⁰ JASCO applies flow matching to temporally controlled text-to-music with both global and local controls, making it highly relevant as a “control injection” reference point for your Intent→Renderer interface. ²¹

Fit to layers: - **Situation:** generally not appropriate (you do not need a generative model to produce a global embedding). - **Intent:** compelling if you view intent as a *stochastic plan* (multiple valid mid-level plans per context). A conditional diffusion over a 60×7 continuous target is feasible and often trains stably, but adds complexity. - **Renderer:** extremely compelling in two roles: 1) **as the renderer itself** (latent diffusion over spectrogram/autoencoder latents), especially if you want strong sample diversity; or 2) **as a decoder/post-filter** that maps imperfect codec tokens to high-fidelity waveform (Multi-Band Diffusion). ²²

The main cost is inference latency (many denoising steps) unless you use accelerated samplers; codec-token AR can be faster for short 6 s segments.

Neural codec token models and hierarchical token pipelines

Neural codecs like SoundStream and EnCodec compress audio into discrete codes via an encoder/decoder + RVQ quantization trained with reconstruction and adversarial losses; DAC (Improved RVQGAN) targets higher fidelity and addresses issues like codebook collapse and bandwidth usage. ²³ These discrete sequences become the modeling space for LMs (AudioLM, MusicLM, MusicGen). ²⁴

Fit to layers: - **Situation:** codec encoders are excellent *acoustic* feature extractors, but are not guaranteed to learn “musical semantics” (style, phrasing) as cleanly as music-specialized representation models; better as part of the conditioning stack than the sole Situation model. - **Intent:** using codec latents/tokens as inputs is attractive because they are already aligned and compress high-rate waveform information; you can build your 10 Hz planner on top of 50–100 Hz codec frames. - **Renderer:** best-supported open path for high-fidelity music generation and controllability, especially when paired with hierarchical generation (semantic/coarse/fine). ²⁵

Best-in-class candidates and layer-by-layer comparison table

The table below summarizes the *ranked* recommendations per SoloMuse layer, followed by detailed specs (architecture, representation, losses, conditioning, and failure modes).

Layer-by-layer comparison table

Layer	Candidate	Core architecture pattern	Representation assumptions	Primary loss	Conditioning mechanism
Situation	Best overall	MERT-style Transformer encoder + attentive pooling + 32-D projection	log-mel/ CQT-derived features inside pretrained model; output is pooled global embedding	downstream-utility fine-tune: contrastive/ MLM-style + auxiliary prediction (optional)	FiLM/adapters into projection head; (optional) multi-task heads
Situation	Best tradeoff	Frozen MuLan (audio tower) or similar music embedding + small projection	pretrained audio embedding space trained with audio-text contrastive objective	MSE/Huber aligning to learned targets + downstream utility	simple concat/ projection; light adapters
Situation	Practical fallback	CRNN/ResNet on log-mel/HCQT + temporal pooling	6 s log-mel / CQT input	supervised regression (MSE) toward “useful” embedding	concat with metadata; dropout
Intent	Best overall	60-query cross-attention planner (Perceiver-IO / encoder-decoder Transformer)	conditioning stream at higher rate: codec frames (50–100 Hz) or spectrogram patches	MSE or Huber + temporal smoothness regularizers	cross-attention + FiLM from situation; condition dropout
Intent	Best tradeoff	Conformer encoder → 10 Hz pooling → MLP head	continuous per-frame audio features	MSE + TV regularization	FiLM + residual adapters
Intent	Practical fallback	BiGRU / TCN over pooled features	60×D input features aligned at 10 Hz	MSE	concat situation, residual MLP

Layer	Candidate	Core architecture pattern	Representation assumptions	Primary loss	Conditioning mechanism
Renderer	Best overall	Hierarchical codec-token seq2seq: coarse→fine acoustic tokens + optional diffusion decoder	neural codec tokens (EnCodec/ SoundStream/ DAC); target is token sequence	cross-entropy over tokens; plus (optional) perceptual decoder losses via diffusion	encoder-decoder cross-attention to backing/ context tokens; FiLM from situation; upsampled intent embedding
Renderer	Best tradeoff	MusicGen-style single-stage Transformer LM on codec tokens	EnCodec-family tokens with codebook interleaving	cross-entropy over interleaved token stream	prefixing or cross-attention; intent injected as time-aligned embeddings
Renderer	Practical fallback	WaveChunkCodec-space diffusion or U-Net regression	keep [599, 882] continuous target	diffusion MSE on noise / or MSE + spectral losses	concat intent+situation; cross-attn optional

Evidence anchors for the above table: codec token LMs for music (MusicGen), hierarchical token pipelines (AudioLM/MusicLM), high-fidelity codecs (SoundStream/EnCodec/DAC), diffusion decoding from tokens (Multi-Band Diffusion), and temporally controlled conditioning (JASCO). ²⁶

Detailed ranked recommendations per layer

Situation layer candidates

Best overall regardless of compute: MERT-based Situation encoder (pooled Transformer) - **Architecture pattern:** Use the pretrained MERT backbone (1D conv front-end + Transformer encoder) and add an attention pooling layer (or CLS pooling if provided) followed by a 2-layer MLP projection to 32 dims. MERT is explicitly designed for music understanding with teacher-guided masked objectives (acoustic teacher + music-specific teacher) to improve music-relevant representations. ²⁷ - **Representation assumptions:** raw audio → MERT internal feature pipeline; output is one pooled embedding per 6 s segment. - **Loss functions:** if you want Situation to be *scientifically meaningful*, don't train it only as "whatever helps the renderer." Instead: - **Stage S1 (frozen backbone):** train only the projection head to maximize downstream predictiveness (Intent/Renderer conditioning utility). - **Stage S2 (light fine-tune):** unfreeze top N layers with a small LR, using a combination of downstream utility + optional self-supervised regularization (e.g., augmentation-invariant contrastive on segment pairs). The teacher-guided design in MERT is specifically motivated by stability and music-relevance; preserve that by keeping fine-tuning gentle. ⁹ - **Conditioning mechanism into downstream models:** treat situation as a **global style/control vector**: inject via FiLM (scale/shift) into Intent and Renderer blocks, and/or via learned prefix tokens. (This is a design choice; your current "broadcast across time" intention aligns naturally with FiLM.) - **Expected failure modes:** (1) embedding collapses to "loudness"/energy unless regularized; (2) embedding learns

shortcuts correlated with train tracks if you still have segment-level leakage (hence track-level splits are mandatory for meaningful evaluation).

Best tradeoff: frozen MuLan audio embedding + projection - **Why:** MuLan's two-tower audio-text contrastive training at very large scale (44M recordings / 370K hours) is explicitly aimed at capturing music semantics and is highly reusable as a global embedding. ²⁸ - **Architecture pattern:** frozen MuLan audio encoder (or a strong open reimplementation if you cannot use the original) + LayerNorm + 32-D projection. - **Loss:** supervise via downstream utility (Intent/Renderer improvements) and optionally a "same-track vs different-track" contrastive probe objective. - **Failure modes:** (1) embedding is too "semantic/text aligned" and misses performance details important for rendering; (2) access/licensing/availability constraints depending on your stack.

Practical fallback: CRNN/ResNet pooled embedding - **Architecture pattern:** log-mel (or HCQT) → 2D CNN (ResNet-ish) → BiGRU/temporal conv → attention pooling → 32-D projection. - **Loss:** purely downstream (predict Intent well; improve Renderer). - **Failure modes:** underfits subtle style and harmonic context, pushing too much burden onto downstream layers.

Intent layer candidates

Best overall regardless of compute: cross-attention planner (Perceiver-IO / encoder-decoder Transformer at 10 Hz) - **Architecture pattern:**

1) Encode the 6 s input audio (or backing audio) into a higher-rate sequence: either codec frames (50–100 Hz) or spectrogram patches.

2) Create **60 learned query tokens** aligned to your 10 Hz grid.

3) Cross-attend queries into the audio token sequence (multi-head cross-attn), followed by temporal self-attn across the 60 queries, then an MLP head to 7 channels. - **Representation assumptions:** You preserve `intent_targets.npy` as continuous [0,1] activations; you do *not* need to quantize yet. - **Loss functions:** baseline MSE is fine (consistent with your smoothing choice), but for stability + calibration: - Use **Huber** (robust to labeling glitches) or MSE with per-channel weighting. - Add a small **temporal total variation (TV) penalty** or second-derivative penalty to reflect the "envelopes rather than impulses" target. - **Conditioning:** Situation [32] injected via **FiLM** into all planner blocks; optionally condition dropout to prevent over-reliance on a single signal (mirrors the "condition dropout supports inference with dropped conditions" idea used in JASCO-style controllable generation). ²⁹ - **Expected failure modes:**

1) **Renderer-bypass:** if the Renderer can infer the "intent" directly from backing audio features, it may learn to ignore the planner. Mitigate by (a) dropping intent during Renderer training some fraction of the time, and (b) evaluating "intent adherence" explicitly.

2) **Oversmoothing:** you may get low MSE but poor responsiveness; monitor calibration curves and thresholded PR-AUC/F1 across multiple thresholds.

Best tradeoff: Conformer planner - **Pattern:** Conformer encoder over audio features with convolutional locality + attention, then downsample/pool to 10 Hz and predict 7 channels per frame. Conformer explicitly targets local+global dependency modeling in audio sequences. ¹⁵ - **Why it's a tradeoff:** easier training than larger cross-attn planners; strong inductive bias; but less explicit separation of "what is observed" vs "what is planned."

Practical fallback: BiGRU/TCN - **Pattern:** project audio features to 10 Hz (by pooling), then BiGRU stack or dilated TCN, then 7-channel head. - **Loss/conditioning:** keep your current MSE; concatenate situation

vector to each time step. - **Failure modes:** weaker long-range phrasing modeling; can become a smoothing filter rather than a planner.

Renderer layer candidates

Best overall regardless of compute: hierarchical codec-token renderer (coarse→fine) + optional diffusion decoder - **Core idea:** switch from deterministic continuous regression to a *probabilistic* model of audio in **discrete codec token space**, then decode to waveform with either a standard codec decoder or a diffusion-based high-fidelity decoder. - **Evidence blueprint:**

- AudioLM explicitly combines semantic tokenization (k-means over SSL model activations) for long-term structure and neural codec tokens for fidelity. ³⁰

- MusicLM adopts a hierarchical sequence-to-sequence pipeline with separate decoder-only Transformers per stage, and explicitly uses coarse→fine acoustic modeling to avoid long sequences. ³¹

- Multi-Band Diffusion is an explicit “decode tokens → high fidelity audio” approach intended to reduce audible artifacts from flawed conditioning and low-bitrate representations. ³² - **Exact architecture pattern (for SoloMuse):**

- **Tokenizer:** EnCodec 24 kHz (or compatible) producing frames at ~50 Hz and K codebooks (MusicGen’s canonical setup: 50 Hz, 4 quantizers, 2048 codebook entries). ³³

- **Condition encoder:** encode backing/context audio into codec tokens too; either feed as an encoder memory or as aligned side tokens. - **Generative core:** Two (or three) decoder stacks: 1) **Coarse stage:** generate codebooks 1..C (e.g., first 1–2 or first 4) autoregressively, conditioned on backing tokens + situation + intent.

2) **Fine stage:** generate remaining codebooks conditioned on coarse outputs + all conditions.

3) Optional “semantic” stage if you later add a higher-level token stream (not required for 6 s, but aligns with AudioLM/MusicLM ideology). ³⁴ - **Loss:** cross-entropy / NLL over discrete tokens; optionally add auxiliary losses on decoded audio (multi-resolution STFT) to stabilize early training, but keep the main objective in token space. - **Conditioning:**

- **Backing/context audio:** encoder-decoder cross-attention (non-causal) or prefix tokens.

- **Intent 60×7:** upsample to token frame rate (e.g., repeat/interpolate from 10 Hz to 50 Hz), embed via MLP, and provide as (a) additive conditioning to token embeddings, and (b) key/value memory in cross-attention.

- **Situation [32]:** FiLM into each block or learned soft prompt tokens. - **Expected failure modes:**

1) **Condition copying / leakage:** if backing audio contains parts of the target (or strongly correlates), the model may “copy” or overfit; MusicGen reports that naive melody conditioning can reconstruct the original sample and introduces an information bottleneck to mitigate overfitting. ³⁵

2) **Codebook imbalance:** higher RVQ levels are harder; coarse-first helps but may reduce fine detail if fine stage is undertrained.

3) **Over-conditioning:** model collapses to deterministic reconstruction, reducing diversity; mitigate via stochastic sampling temperature and/or conditional dropout.

Best tradeoff: MusicGen-style single-stage Transformer LM over codec tokens - MusicGen’s core claim is that a **single-stage Transformer LM** with carefully designed codebook interleaving can reach high quality without a complex cascade. ³⁶ - MusicGen’s EnCodec configuration is explicit: non-causal 5-layer EnCodec at 32 kHz, stride 640 → 50 Hz, RVQ with 4 quantizers of size 2048. ³⁷ For SoloMuse you would likely use the 24 kHz EnCodec model instead to avoid resampling, but keep the same conceptual “50 Hz × codebooks” framing. ³⁸ - Critical nuance: MusicGen shows “flattening” can score best but is costly, and notes similar performance may be reached at a fraction of the cost with a delay pattern. ³⁹

For SoloMuse, token count per 6 s is modest, so you can simply choose the pattern that yields best adherence. - **Conditioning:** your conditions (backing, intent, situation) slot into the same interface

MusicGen uses for text/melody; the “information bottleneck” idea used for melody conditioning is directly applicable to prevent “just reconstruct the conditioning input.” ³⁵

Practical fallback: stay in WaveChunkCodec space but switch to diffusion/U-Net - If you cannot afford a representation swap immediately, the biggest weakness of the current Conv1D+MSE renderer is that it behaves like a deterministic regressor in a multimodal domain (music “has many valid realizations”). Diffusion models were introduced partly to provide high-quality generation with better sample diversity than simple regression, and have established audio-synthesis successes (DiffWave/WaveGrad). ⁴⁰ - A WaveChunkCodec-space diffusion renderer is a reasonable bridge: keep your targets, but train a conditional diffusion model that predicts noise in your 599×882 representation, conditioned on backing+intent+situation. If it works, you can later swap the diffusion “data space” to codec latents/tokens.

Integrated final stack recommendation

This section proposes a coherent end-to-end SoloMuse stack (best-in-class) and a practical fallback, plus training order and how to decide whether to upgrade intent representation.

Best-in-class SoloMuse stack

Key design decision: switch Renderer target to codec tokens.

Open research consensus for high-fidelity music generation increasingly centers on codec token modeling (Jukebox → AudioLM/MusicLM → MusicGen), using learned discrete representations to reduce sampling rate while preserving fidelity. ⁴¹ Your current [599,882] continuous target is workable for reconstruction, but it is not the representation family most modern music generation literature builds on for controllable high-quality synthesis.

Codec choice recommendation (for your 24 kHz, 6 s setup): - Default: **EnCodec 24 kHz** for straight compatibility and lower token rates; EnCodec is explicitly designed as a high-fidelity neural codec with training stabilizers (e.g., loss balancing). ⁴² - Strong alternative: **DAC (Improved RVQGAN)** if you are willing to resample and accept higher token rates; DAC is designed to improve fidelity and codebook usage and discusses issues like codebook collapse. ⁴³

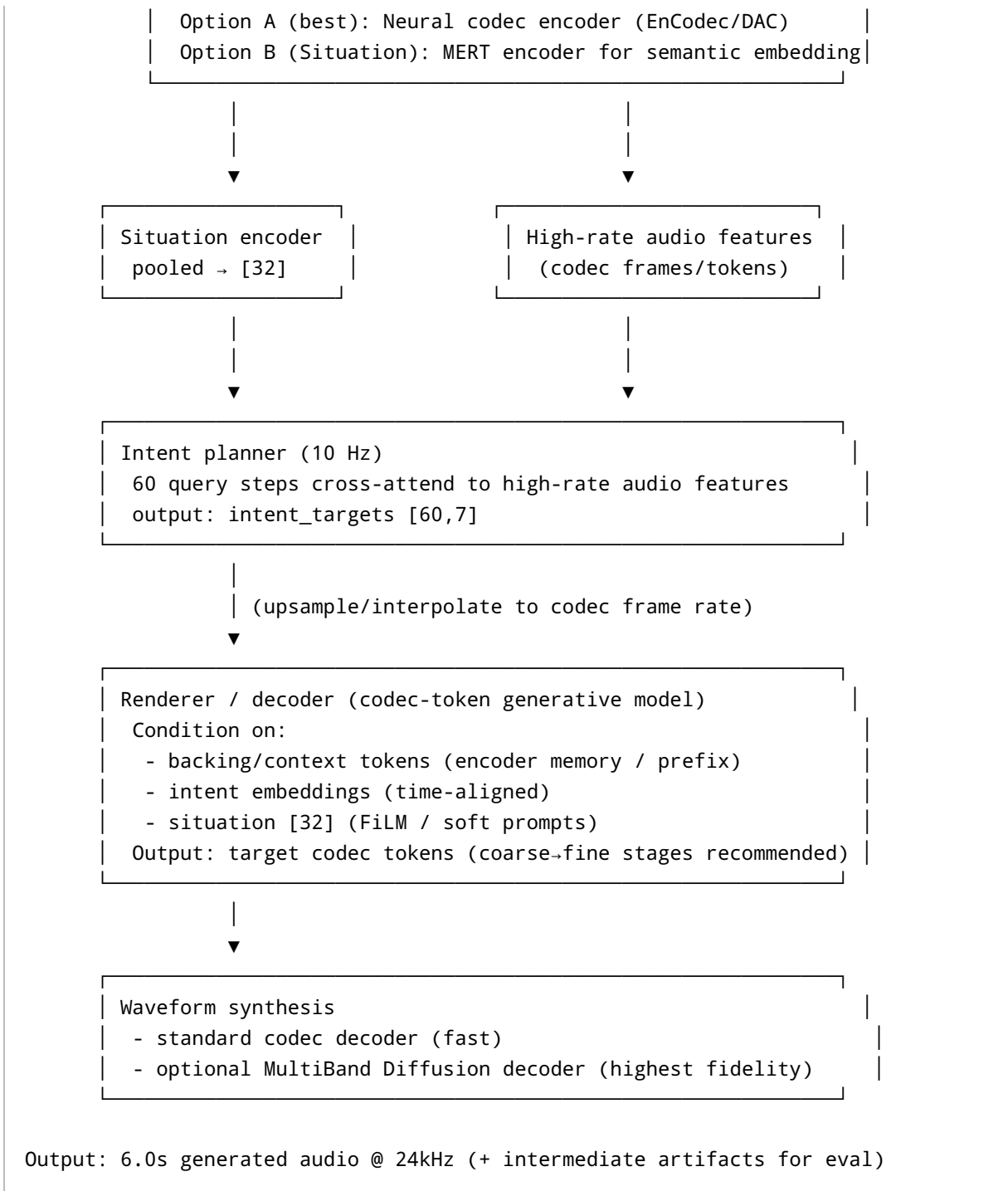
- Practical evidence point: MusicGen reports that using DAC vs EnCodec changes FAD/KL/subjective behavior and that EnCodec’s lower frame rate (50 Hz vs DAC 86 Hz in their setup) reduces runtime; they also caution tokenizer fitness needs task-specific experimentation. ⁴⁴

Renderer decoding upgrade: Keep a “premium decode” flag that runs **Multi-Band Diffusion** to decode tokens when you want maximal fidelity; Multi-Band Diffusion is explicitly positioned as reducing audible artifacts compared to standard generative decoding at equal bitrate, at the cost of extra compute. ¹⁹

Information flow and stack diagram

Input(s): 6.0s backing/context audio @ 24kHz (+ optional other context channels)

Shared audio tokenizer/encoder



This design deliberately aligns SoloMuse with the dominant open “codec tokens + generative model” line (AudioLM/MusicLM/MusicGen), while retaining your core controllability principle: **Situation is global** (broadcast) and **Intent is local** (10 Hz plan). ⁴⁵

Training order and joint fine-tuning

A scientifically clean training strategy is **hierarchical pretraining** → **staged integration** → **limited joint fine-tuning**, specifically because each layer can otherwise learn shortcuts.

- Train (or adopt pretrained) **tokenizer/codec** first and freeze it. This mirrors essentially all codec-token pipelines. ⁴⁶
- Train **Situation** as a frozen-backbone projection initially; only later allow partial fine-tuning if you can demonstrate robust generalization under track-level splits (otherwise Situation will memorize). ⁴⁷
- Train **Intent** next, using ground-truth intent targets (your stable baseline objective), and validate that it predicts well *without* renderer feedback.
- Train **Renderer** with Intent teacher forcing (use the ground truth intent plan during early training), then switched conditioning (mix of predicted intent and ground truth) to make the system robust to planner errors.
- Optional: small-LR joint fine-tune of (Intent + Renderer) while keeping Situation mostly fixed. The joint fine-tune should be justified by an ablation that shows measurable end-to-end gains without collapsing interpretability.

Should intent remain framewise [60,7] or become event/token based?

Your current `intent_targets.npy` is effectively a *soft control envelope* over 6 seconds. Keeping it framewise is defensible in the near term because: - it is already aligned, smooth, and stable to optimize with MSE (your baseline virtue), and - it is expressive enough to drive controllability if the renderer is forced to depend on it (via dropout and evaluation).

That said, if “musical phrasing” is a top goal, the strongest medium-term upgrade is to **add a discrete plan bottleneck in parallel**, not necessarily to discard the [60,7] representation: - learn a small **VQ-VAE over the 60×7 plan** to produce a short sequence of “phrase tokens” (e.g., 15–30 tokens over 6 s) and condition the Renderer on both the continuous envelope and the discrete plan. This mirrors the general success of discrete token hierarchies in AudioLM/MusicLM. ⁴⁸ - JASCO is valuable inspiration here: it is explicitly built for temporally controlled music generation and emphasizes integrating global and fine-grained local controls robustly. ⁴⁹

Evaluation plan

A rigorous evaluation plan for SoloMuse must (1) eliminate leakage, (2) measure each layer *on its own terms*, and (3) measure end-to-end controllability and musical quality with both objective metrics and human studies.

Split hygiene

Track-level grouped splits are non-negotiable. Segment-level splits will leak timbre, mixing signatures, themes, and even near-duplicate material across train/val/test, making your “layer improvements” scientifically unreliable (especially for Situation embeddings).

Concrete split protocol: - Assign a unique `track_id` for every 6 s segment. - Perform grouped splitting so no `track_id` appears in more than one split. - Reserve a “hard test” set of artists/instrumentation/mixing

archetypes if possible (even if only a subset), because controllable generation often fails under distribution shift.

Metrics by layer

Situation - Downstream utility (primary): measure delta in Intent and Renderer performance when Situation is present vs shuffled vs dropped. This directly tests whether Situation captures global context in a useful way. - **Representation stability checks:**

- same track, different segments: embedding similarity distribution;
- augmentation invariance: small pitch/time stretch perturbations should not destroy high-level embedding unless you *want* it to encode absolute key/tempo (decide explicitly). - If you have tempo/key/genre tags, add lightweight probes; if not, focus on downstream utility.

Intent - Regression: MSE and MAE per channel (base), but also report *time-weighted* metrics emphasizing transitions if phrasing matters. - **Thresholded behavior:** because targets are soft [0,1], compute PR-AUC or F1 across multiple thresholds (e.g., 0.2/0.5/0.8) to detect whether the model is merely regressing toward means. - **Calibration:** reliability curves (bin predicted intent vs empirical occupancy above threshold) and Brier score; this matters because the Renderer will treat intent as control.

Renderer Split evaluation into two regimes: **reconstruction** (match ground truth under teacher-forced intent) and **generation** (sample quality + diversity).

Objective audio metrics: - **FAD (Fréchet Audio Distance)** is a widely used reference-free metric originally proposed for music/audio quality evaluation in embedding space. ⁵⁰

- Use an adaptation toolbox/strategy designed for generative music where possible (FAD's correlation depends on embeddings and the reference set). ⁵¹ - Spectral metrics (multi-resolution STFT distance, log-mel L1) for reconstruction diagnostics (do not treat as "musicality").

Human evaluation (end-to-end and renderer-only): - **MUSHRA** (ITU-R BS.1534) is a standard protocol for subjective assessment of intermediate audio quality and is explicitly used in codec evaluation literature like EnCodec; use it for renderer quality and artifact detection. ⁵²

- Pairwise preference tests specialized for controllability: raters judge (a) musicality, (b) coherence with backing, and (c) adherence to intent constraints.

End-to-end - Condition adherence tests: deliberately manipulate intent channels (hold one channel high/low, swap intent between segments) and measure whether the audio changes accordingly. - **Ablation-driven success:** define end-to-end success as simultaneously improving: 1) human preference or MUSHRA score, and

- 2) intent adherence metrics, without collapsing diversity.

Ablations (minimal set that yields strong causal conclusions)

- 1) **No Situation vs Situation:** drop or shuffle situation at inference; measure intent/renderer deltas.
- 2) **No Intent vs Intent:** train renderer with intent dropped 100% vs 0% vs scheduled dropout.
- 3) **Intent representation:** continuous [60,7] vs (continuous + learned discrete tokens).
- 4) **Renderer target:** WaveChunkCodec continuous vs codec tokens (EnCodec vs DAC). Tokenizer

comparisons are known to affect results in MusicGen. ⁴⁴

5) **Frozen vs joint fine-tuning:** freeze Situation+Intent vs jointly fine-tune; watch for “planner ignored” collapse.

What success looks like

Small scale (first 1–2 weeks of serious experiments): - Track-split correctness verified. - Clear measured benefit from adding Situation (or a clear conclusion it is redundant). - Renderer upgrade shows objective improvements (FAD down, artifacts down) *and* human preference trends upward in small tests.

Full scale (after architecture stabilizes): - End-to-end system can intentionally vary phrasing behaviors (as defined by your 7 channels) without losing overall musicality, and perceptual quality improves under standard listening tests (MUSHRA/pairwise). ⁵³

Implementation roadmap

This roadmap is designed to preserve your current stability while moving toward best-in-class token modeling. Each phase lists prerequisites, code changes, training recipe, validation checks, and expected artifacts.

Phase 1: split hygiene and evaluation harness

Prerequisites: reliable `track_id` mapping for every 6 s segment; deterministic data loading.

Code changes required:

Implement grouped splitting by `track_id`; log split statistics (tracks, segments, duration); ensure reproducible seeds and that no track overlaps.

Training recipe: run your existing baseline models unchanged, but under track splits; establish baseline curves.

Validation checks:

- hash-based overlap test (no shared track IDs);
- verify distribution shift (genre/tempo proxies if you have them) is not extreme unless intended;
- sanity checks: decode a few renderer outputs exactly as you already do.

Expected artifacts:

- `splits.json` with track IDs;
- baseline metric report for each layer (Situation/Intent/Renderer) under track split;
- a “leakage alarm” unit test run in CI.

Phase 2: strongest low-risk architecture upgrade

Goal: upgrade *one layer* with maximal benefit and manageable surface area.

Recommended low-risk priority: **Renderer target swap to codec tokens + small Transformer**, because it aligns you with modern pipelines and lets you reuse established recipes. MusicGen is explicit about an EnCodec configuration (50 Hz, multi-codebook RVQ) and codebook interleaving patterns that directly matter for training/inference. ⁵⁴

Prerequisites:

- choose codec (start EnCodec 24 kHz);
- build a token caching/precompute step (many token-LM stacks precompute codec tokens for efficiency; AudioCraft explicitly supports token-based workflows). ⁵⁵

Code changes required:

- new dataset field `renderer_tokens.npy` (e.g., `[T, K]` integer tokens);
- renderer model switched from Conv1D regression to Transformer LM;
- decode pipeline: tokens → waveform using codec decoder.

Training recipe:

- Start with ~300M-class model scale (MusicGen trains multiple sizes; use smaller first for iteration). ⁵⁶
- Teacher forcing; scheduled sampling later; intent conditioning uses ground truth initially.

Validation checks:

- token decode round-trip quality (codec reconstruction), separate from generation;
- overfit test on a tiny subset (should learn quickly);
- verify the model can't trivially reconstruct by copying conditioning (use condition dropout + swap tests).

Expected artifacts:

- tokenized dataset cache;
- `renderer_transformer.pt`;
- audio samples decoded from predicted tokens.

Phase 3: best-in-class final model

Goal: full controllable stack with high fidelity.

Prerequisites: stable token renderer; intent planner shows predictive power and isn't ignored.

Code changes required:

- Introduce **hierarchical rendering** (coarse→fine). MusicLM explicitly describes splitting acoustic modeling into coarse and fine stages (first 4 RVQ levels vs remaining levels) to manage token lengths and quality. ³¹
- Optional: add **Multi-Band Diffusion** decoding option to reduce codec artifacts. ¹⁹
- Upgrade Intent planner to cross-attention if not already.

Training recipe:

- Stage-wise: train coarse stage to convergence; freeze or partially freeze; train fine stage; then joint fine-tune.
- Use explicit "intent adherence" evaluation during training (don't wait for end).

Validation checks:

- ablations (intent present vs dropped) show measurable change;
- human mini-tests for controllability and audio artifacts.

Expected artifacts:

- two (or more) renderer checkpoints (coarse / fine);
- optional diffusion decoder checkpoint;
- reproducible evaluation suite outputs.

Phase 4: scaling and optimization

Goal: scale quality and robustness; reduce latency if needed.

Prerequisites: best-in-class stack is clearly better than baseline on track-split metrics and human eval.

Code changes required:

- distributed training/efficient attention (MusicGen uses memory-efficient attention implementations; keep your stack compatible). ⁵⁷
- optionally experiment with a state-space backbone (Mamba) for longer sequences if you later expand beyond 6 s, but treat this as higher uncertainty relative to Transformer baselines. ⁵⁸

Training recipe:

- scale model size and data; increase conditioning diversity; add hard out-of-domain tests.

Validation checks:

- regression on older benchmarks;
- ensure controllability doesn't collapse at scale.

Expected artifacts:

- large-scale checkpoints;
- updated model cards and reproducibility logs.

Risks and unknowns, plus what to prototype first this week**Risks and unknowns**

Intent might be redundant unless you force usage. If the Renderer can infer “what to do” directly from backing audio tokens (or if your targets are strongly predictable), the planner can be ignored. You should assume this will happen unless you implement (a) intent dropout, (b) intent swap tests, and (c) intent adherence metrics.

Representation swap risk (WaveChunkCodec → codec tokens). Tokenizers can change what information is preserved; MusicGen explicitly notes tokenizer choice (EnCodec vs DAC) impacts results and requires experimentation. ⁴⁴

Codec training/usage stability. Codebook collapse and adversarial training instability are real; DAC discusses codebook collapse and fixes, and EnCodec introduces stabilizers like loss balancing. ¹²

Evaluation mismatch. FAD is useful but embedding/reference choices can affect correlation; use the “adapted FAD for generative music” guidance and always ground with listening tests (MUSHRA). ⁵⁹

6-second context ceiling. Many music structure phenomena live beyond 6 s. Token models (MusicLM) explicitly target multi-minute consistency, but they rely on long training sequences and multi-stage designs. ⁶⁰ If SoloMuse stays at 6 s, you should define “phrasing” and “control” within that horizon, or plan a future “long context” extension.

What to prototype first this week

If I had to choose the highest-leverage prototypes that fit into one week of focused engineering:

1) **Track-level split migration + leakage tests (day 1–2).** This is the foundation; everything else is scientifically noisy without it.

2) **Renderer tokenization spike (day 2–3).** Add a script that converts each 6 s segment into codec tokens (start with EnCodec 24 kHz), saves `renderer_tokens.npy`, and can decode them back to waveform for sanity checks. The goal is not “best model,” it’s proving the new artifact pipeline is shape-stable and decodes correctly.

3) **Small conditional token Transformer renderer (day 3–5).** Train a small model that predicts target tokens conditioned on backing tokens + ground-truth intent + situation. Even a tiny model should overfit a small subset quickly. This is your fastest signal on whether “token space rendering” will outperform WaveChunkCodec regression.

4) **Intent-adherence unit tests (day 5–7).** Add two deterministic tests: - **Intent swap:** swap intent plans between two examples with similar backing, see if generation changes in the intended direction. - **Intent ablation:** drop intent entirely and measure whether output collapses or remains similar. These tests prevent you from shipping a pipeline where the “planner” is effectively unused.

The above “week one” sequence is designed to lock in scientific rigor (splits + tests) while moving directly toward the dominant high-quality architecture family (codec tokens + conditional generative model). ⁶¹

¹ ²³ ⁴⁶ <https://arxiv.org/abs/2107.03312>

<https://arxiv.org/abs/2107.03312>

² ⁷ ¹⁰ ¹¹ ¹³ ²⁶ ³³ ³⁵ ³⁷ ³⁹ ⁴⁴ ⁵⁴ ⁵⁶ ⁵⁷ ⁶¹ <https://arxiv.org/pdf/2306.05284>

<https://arxiv.org/pdf/2306.05284>

³ ⁸ ²⁵ ³¹ <https://arxiv.org/pdf/2301.11325>

<https://arxiv.org/pdf/2301.11325>

⁴ ⁹ ²⁷ ⁴⁷ <https://arxiv.org/pdf/2306.00107>

<https://arxiv.org/pdf/2306.00107>

5 21 49 <https://arxiv.org/abs/2406.10970>

<https://arxiv.org/abs/2406.10970>

6 18 40 <https://arxiv.org/abs/2009.09761>

<https://arxiv.org/abs/2009.09761>

12 43 <https://arxiv.org/pdf/2306.06546>

<https://arxiv.org/pdf/2306.06546>

14 29 <https://huggingface.co/facebook/jasco-chords-drums-melody-400M>

<https://huggingface.co/facebook/jasco-chords-drums-melody-400M>

15 https://www.isca-archive.org/interspeech_2020/gulati20_interspeech.html

https://www.isca-archive.org/interspeech_2020/gulati20_interspeech.html

16 <https://arxiv.org/abs/2111.00396>

<https://arxiv.org/abs/2111.00396>

17 58 <https://arxiv.org/abs/2312.00752>

<https://arxiv.org/abs/2312.00752>

19 22 32 <https://arxiv.org/abs/2308.02560>

<https://arxiv.org/abs/2308.02560>

20 <https://arxiv.org/abs/2407.14358>

<https://arxiv.org/abs/2407.14358>

24 30 34 45 48 <https://david.grangier.info/papers/2023/audio-lm-generation.pdf>

<https://david.grangier.info/papers/2023/audio-lm-generation.pdf>

28 <https://arxiv.org/pdf/2208.12415>

<https://arxiv.org/pdf/2208.12415>

36 Simple and Controllable Music Generation

https://arxiv.org/abs/2306.05284?utm_source=chatgpt.com

38 <https://audiocraft.metademolab.com/codec.html>

<https://audiocraft.metademolab.com/codec.html>

41 <https://arxiv.org/abs/2005.00341>

<https://arxiv.org/abs/2005.00341>

42 <https://arxiv.org/abs/2210.13438>

<https://arxiv.org/abs/2210.13438>

50 https://www.isca-archive.org/interspeech_2019/kilgour19_interspeech.html

https://www.isca-archive.org/interspeech_2019/kilgour19_interspeech.html

51 59 <https://arxiv.org/html/2311.01616v2>

<https://arxiv.org/html/2311.01616v2>

52 53 https://www.itu.int/dms_pubrec/itu-r/rec/bs/R-REC-BS.1534-3-201510-I%21%21PDF-E.pdf

https://www.itu.int/dms_pubrec/itu-r/rec/bs/R-REC-BS.1534-3-201510-I%21%21PDF-E.pdf

55 <https://huggingface.co/spaces/facebook/MusicGen/blame/main/docs/MUSICGEN.md>

<https://huggingface.co/spaces/facebook/MusicGen/blame/main/docs/MUSICGEN.md>

⁶⁰ [2301.11325] MusicLM: Generating Music From Text
https://arxiv.org/abs/2301.11325?utm_source=chatgpt.com